



PDF Download
3641525.3663624.pdf
04 February 2026
Total Citations: 2
Total Downloads: 219

Latest updates: <https://dl.acm.org/doi/10.1145/3641525.3663624>

RESEARCH-ARTICLE

Towards Evidence-Based Software Quality Practices for Reproducibility: Practices and Aligned Software Qualities

CHRISTIAN GILBERTSON, Sandia National Laboratories, Albuquerque, NM, United States

MIRANDA R MUNDT, Sandia National Laboratories, Albuquerque, NM, United States

JOSHUA B TEVES, Sandia National Laboratories, Albuquerque, NM, United States

SIMONE TORIBIO, Sandia National Laboratories, Albuquerque, NM, United States

REED M MILEWICZ, Sandia National Laboratories, Albuquerque, NM, United States

Open Access Support provided by:

Sandia National Laboratories

Published: 18 June 2024

[Citation in BibTeX format](#)

ACM REP '24: ACM Conference on
Reproducibility and Replicability
June 18 - 20, 2024
Rennes, France

Conference Sponsors:
EIGREP

Towards Evidence-Based Software Quality Practices for Reproducibility: Practices and Aligned Software Qualities

Christian Gilbertson

cjgilbe@sandia.gov
Sandia National Laboratories
Albuquerque, New Mexico, US

Miranda Mundt

mmundt@sandia.gov
Sandia National Laboratories
Albuquerque, New Mexico, US

Joshua Teves

jbteves@sandia.gov
Sandia National Laboratories
Albuquerque, New Mexico, US

Simone Toribio

sstorib@sandia.gov
Sandia National Laboratories
Albuquerque, New Mexico, US

Reed Milewicz

rmilewi@sandia.gov
Sandia National Laboratories
Albuquerque, New Mexico, US

ABSTRACT

Among computational science and engineering (CSE) software development projects, reproducibility of results is widely understood to be essential for the software to be useful and meaningful. At present, there is a lack of empirical support for how best to design for, implement, or maintain reproducibility over the course of the CSE software lifecycle, and the relationship between reproducibility and other qualities of interest is not well-understood. In this study, we consider the role of software engineering practices and product qualities in enabling reproducibility. To build a foundation for future studies, we conducted case studies of the software engineering practices of 9 teams at Sandia National Laboratories (Sandia), and a first-in-kind online survey of 219 developers and users of CSE software to assess their beliefs about quality and reproducibility. Among our results, we identified three practices that were universally attributed to enhancing reproducibility of research and software at Sandia: automated testing, documentation, and version control. We found that the majority of survey respondents believed that high-quality software was more likely to be reproducible than low-quality software, and that maintainability, reliability, and usability were generally seen as positively contributing to reproducibility. These preliminary findings merit follow-up studies to better understand how software quality is valued and enacted in CSE software development practice.

ACM Reference Format:

Christian Gilbertson, Miranda Mundt, Joshua Teves, Simone Toribio, and Reed Milewicz. 2024. Towards Evidence-Based Software Quality Practices for Reproducibility: Practices and Aligned Software Qualities. In *ACM Conference on Reproducibility and Replicability (ACM REP '24)*, June 18–20, 2024, Rennes, France. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3641525.3663624>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ACM REP '24, June 18–20, 2024, Rennes, France
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0530-4/24/06
<https://doi.org/10.1145/3641525.3663624>

1 INTRODUCTION

Today, computing plays a central role in the practice of science and engineering. Our prosperity, security, and ability to understand the world in which we live increasingly depends upon computational science and engineering (CSE) software. For this reason, ensuring the quality of that software is a matter of great importance. On the other hand, teams must choose how to allocate finite development resources. Reproducibility is only one of many qualities to be balanced for a team's piece of software, and its strategies will change as the software lifecycle progresses. Adding reproducibility during requirements gathering or software specification is very different from trying to incorporate it after the software has already begun maturing and is harder to modify. Existing literature provides little advice on the best way to balance reproducibility with competing software qualities, preventing researchers and developers from making evidence-based decisions for incorporating and improving reproducibility in their software projects. Cultivating a better understanding of the needs of CSE teams will help us develop more effective solutions for managing software qualities across development lifecycles.

Seth et al. [66] noted in their sample of 11 software companies that there were deliberate actions taken by various stakeholders to:

- define the objectives of software development,
- identify the necessary quality characteristics, and
- set strategies to create such quality in the software.

In the absence of the stakeholders prioritizing a given quality, it is likely that the software's objectives, characteristics, and strategies

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525. This work was supported by the Laboratory Directed Research and Development (LDRD) program (grant 229304).

Author Credit: Christian Gilbertson assembled the text for this manuscript, created the codebook used for coding the case study interviews, aided in coding the interviews and performed analysis of the survey results. Reed Milewicz was a co-PI and led the survey design and case study interviews and performed analysis of the survey results. Miranda Mundt was the second co-PI and handled project management considerations and data curation, led the case study design and preliminary research activities, and aided in the coding of the case study interviews. Joshua Teves aided in the coding of the case study interviews and the assembly of this manuscript. Simone Toribio aided in survey design.

will neglect it. This creates a fundamental challenge for development teams to consider reproducibility if it is not emphasized by their stakeholders.

In general, poor software quality creates barriers to reproducibility, and software of higher quality is more likely to remain reproducible (and easier to repair, should reproducibility degrade) [60]. This has resulted in a prevailing belief that better software development practices promote better quality software, which then improves the reproducibility and reusability [24].

However, the exact relationship between reproducibility and other software qualities of interest is unclear. The most reproducible software may be less performant, less reusable, or more expensive to maintain, and teams need guidance in order to effectively evaluate these trade-offs. Likewise, the connection between reproducibility and software practices is also undetermined. With all this in mind, we arrived at the following research questions:

- (1) **RQ1:** What software development practices facilitate the development and use of reproducible software?
- (2) **RQ2:** Do CSE developers believe software quality and reproducibility are related? Do they feel willing and able to improve the quality of their software to benefit reproducibility?
- (3) **RQ3:** To what extent are different software product quality characteristics perceived to contribute to reproducibility? Do CSE projects prioritize these software product quality characteristics in their work?

To answer these questions, we built off of our previous work [53] and conducted case studies of 9 CSE teams at Sandia National Laboratories, a United States national laboratory, about the practices they use to keep their software reproducible and a survey of 219 CSE software developers inside and outside of Sandia National Laboratories about reproducibility's relationship to other software qualities. The case studies revealed 7 categories of practices commonly used by teams to enable computational reproducibility: automated testing, code reviews, having a development pipeline, documentation, having public processes, team communication, and version control. The studied teams universally highlighted automated testing, documentation, and version control as useful for reproducibility. Our survey found clear positive correlations between reproducibility and (1) maintainability, (2) reliability, and (3) usability. It also found clear negative correlations between reproducibility and (1) security and (2) performance efficiency. The survey also revealed a lack of understanding and expertise in respondents on how to achieve reproducibility, as well as the belief that stakeholders underestimate the time and effort required. The rest of the paper is organized as follows: Section 2 presents relevant background for the problem area. Sections 3 and 4 discuss our methodology and results. Section 5 presents possible threats to validity, and Section 6 provides an analysis of the findings.

2 BACKGROUND

In this paper, we use the term CSE software to refer to any software that supports the exploration of a science or engineering question, ranging from low-level I/O and accelerator libraries to sophisticated software stacks for simulation, data analysis, and visualization. Investments in CSE regularly yield transformative impacts in diverse

areas such as energy, healthcare, climate science, and national security; to quote one think tank, “the impact of [high-performance computing for science and engineering] touches almost every aspect of daily life [...] In short, advanced computing increasingly determines a nation's economic as well as defense security” [23]. As the demand for CSE software has grown, so too has the community evolved to meet that demand. While historically there has been only limited communication between CSE community and the world of conventional software engineering (see [44]), in the past decade we have seen the community embrace open-source and community-driven software development [10, 12, 17, 33, 42, 43], the emergence of the research software engineer (RSE) as a distinct professional role in CSE [5, 13, 18], and widespread interest in adapting software engineering best practices to CSE contexts [2, 21, 32, 48, 61]. For software engineering researchers, this new era of cooperation presents opportunities for constructive dialogue with the CSE community.

Among the many topics surrounding CSE software development which deserve more scholarly attention, reproducibility stands out as a necessary condition for CSE products. At present, there are many different competing definitions of reproducibility across disciplines [31]; for example, the ACM has proposed separate definitions for repeatability, reproducibility, and replicability which distinguish who is running the computational experiment (same team or different team) and whose software and hardware is being run (same experimental setup or with independently developed software) [1]. For the purposes of our study, we use the definition of reproducibility given by the National Academies of Sciences, Engineering, and Medicine (NASEM) in 2019, which is “the ability to obtain consistent results from software using the same input data, computational steps, methods, and code, and conditions of analysis” [56], which can be read as broadly encompassing all three of the ACM definitions [56]. In general, when we say reproducibility, we mean that when someone uses a piece of CSE software to answer a significant science or engineering question, they need to be able to get consistent results from that software now and in the future. We seek to understand reproducibility as a quality of scientific software, and what is needed to ensure it is at the desired levels, from developers and their stakeholders. We leave other matters (e.g., what qualities as consistent, who carries out the task of reproducing results) open-ended.

Various researchers have extended ISO/IEC 25010:2011 and other quality standards to accommodate novel attributes of interest such as greenability [14], explainability of AI [47], user satisfaction for e-learning tools [3], efficiency of semantic web exploration tools [29], and playability of serious games [25]. Methodologically speaking, the most similar work in this space to ours is that of Condori-Fernandez and Lago, who conducted a survey to identify what characteristics of software quality were perceived as contributing to sustainability – this study provided the inspiration for our survey [19]. Previous research has explored the application of software quality standards to scientific software development [46, 52], but to our knowledge ours is the first to explore the relationship between reproducibility and different software quality characteristics. There have, however, been studies linking quality with reproducibility in a broader sense. Pimental et al. investigated the reproducibility

of Jupyter notebooks, and found that only 24% out 1,053,653 notebooks studied remained executable, and only 4% produced results consistent with their original outputs [59]. They noted how factors like lack of built-in testing, poor design and usability (e.g., ambiguous and out-of-order cell executions, inadequate documentation), and undeclared or un-versioned dependencies contribute to the low rate of reproducibility among notebooks. Likewise, Trisovic et al. found that while 74% out of 9000 R scripts published as part of replication datasets between 2010 and 2020 failed to run without errors, 18% of these could be fixed with automatic code cleaning routines, indicating that good coding practices can help ensure reproducible results [70]. Finally, Grayson et al. considered the reproducibility of Snakemake and Nextflow workflows [30], finding that the median survival time of a workflow (from the point at which it was created to the point at which the workflows can no longer produce the original results) is 1.42 years. Our focus is distinct from the studies linking quality with reproducibility in a broader sense as we are directly investigating the link between any given software quality and reproducibility for scientific software, as well as the software development practices that contribute to this reproducibility.

Computational science has a rich and storied tradition in the theory and practice of reproducible and verifiable computation. As Lin and Zhang have noted, reproducibility should be seen as an ongoing process rather than a one-time achievement – the product of a commitment to practices, processes, and norms that promote it [50]. In the past decade, the scientific community has made significant strides in this direction. We have seen a culture shift towards embracing open-source sharing of code and data [4, 37], maintaining provenance and traceability from inputs to publications [24, 49, 65], and aligning work with policies from funders and journals [71, 72]. Meanwhile, from a tooling and infrastructure perspective, teams are increasingly using containers to preserve computational environments [9, 57], workflows and notebooks to capture all the computational steps involved [6, 20, 45], and online platforms that promote standardization and sharing of these supporting artifacts [16, 28, 67, 68]. These solutions, however, emphasize a “publication pipeline” mindset in which the software artifacts are published, curated, and made reproducible when the science is complete.

While these are all useful advances, we observe that these solutions are mainly *post-hoc* methods after the development of the scientific software is “complete”. As Hocquet and Wieber have noted, the role of software itself and its development in ensuring computational reproducibility have received relatively little attention [35]. To enrich the discussion, in our work we examine reproducibility through the lens of software quality and practices. At present, there are no software quality standards in industry or academia that account for reproducibility as a quality of software (*cf.*, [73]).

3 METHODOLOGY

First, we aimed to identify software development practices that support reproducibility across the software lifecycle. Development teams intentionally use practices, processes, and tools that help them achieve their quality goals [62]. Along similar lines, there are practices that teams are doing - or could be doing - at different

stages of their work to make their software and its results more reproducible.

Second, we aimed to quantify and qualify software quality trade-offs between reproducibility and other aspects of quality. As noted in Section 1, in general, poor software quality creates barriers to reproducibility, and high-quality software is more likely to remain reproducible, and it becomes easier to repair this quality as it degrades. At the same time, the most reproducible software may be less performant, less reusable, or more expensive to maintain, and teams and their customers need better guidance on how to navigate those trade-offs.

The data that support the findings of this study (including the survey instrument, codebook, interview questions and analysis code) are openly available on Zenodo¹.

3.1 Case Study of Reproducibility Practices at Sandia

A case study is “[a]n empirical enquiry that draws on multiple sources of evidence to investigate one instance (or a small number of instances) of a contemporary software engineering phenomenon within its real-life context, especially when the boundary between phenomenon and context cannot be clearly specified” [64]. Due to the fact that reproducibility and its relationship to other qualities is understudied, it would be difficult to remove some phenomena from their contexts, making a case study a suitable approach. We conducted an explanatory case study (see [64]) to identify software quality practices that scientific teams take with respect to reproducibility and practices by other stakeholders to exercise those qualities in use. To mitigate the risk of oversampling and to gain a more holistic view of reproducibility practices and stakeholder experiences, we recruited an ensemble of different stakeholders for our case study.

When assembling this ensemble, we partnered with a range of individuals and teams at Sandia National Laboratories spanning both the scientific software supply chain (from developers to analysts to decision-makers to quality consultants) and different domains (renewable energy, climate science, machine learning, credibility processes, and nuclear weapon development). Each team was represented by two or more, dependent on team size, team members in the interviews. A high-level summary of all projects can be seen in Table 1. Here and elsewhere, we use pseudonyms to protect the privacy of our interviewees. We sought to collect data on what strategies real-world teams were using to achieve reproducibility and how that interacted with other qualities of interest for software development. This consisted of three interviews. The first interviews focused on data gathering on the project and establishing the project’s main purpose and practices. The second interviews focused on data gathering around specific software engineering and reproducibility practices. These two interviews were then synthesized to help direct a supplemental, third interview where teams selected a specific practice and made a Progress Tracking Card (PTC)² [54] to determine how to make progress towards their reproducibility goals.

¹<https://zenodo.org/records/11150653>

²PTCs are part of the Productivity and Sustainability Improvement Planning framework [34]: a lightweight, iterative process improvement workflow that allows software

Table 1: Overview of projects recruited to participate in our case study at Sandia National Laboratories.

Project	Purpose	Size of Team	Age (years)
Project A	Data processing toolkit for marine energy science	12	4
Project B	Machine learning for natural language processing	7	4
Project C	Monte Carlo radiation transport	3	3
Project D	Input deck generation for thermal fluids simulations	6	1
Project E	Nonlinear optimization algorithms for marine energy	5	1
Project F	Wave energy converter design	9	10
Project G	Water systems resilience	5	8
Project H	GPS software development	8	6
Project I	Mathematical optimization modeling	6	15

We hypothesized that scientific software stakeholders value both software quality and reproducibility, but they may or may not have a clear understanding of the connection between the two. We used the case study transcripts to answer the question of what software practices teams use across the software lifecycle with Grounded Theory[27]. We first identified noteworthy excerpts from the transcripts and grouped them together with related excerpts (falling under a given code such as “Practice Advice” or “Barriers and Mitigation”) with the constant comparative method, resulting in a codebook. We then grouped related codes together into super codes (such as “Team Description” or “Quality and Reproducibility”) with axial coding. This coding revealed (among other things) categories of practice the teams used to enhance their reproducibility. These interviews provided an evidentiary basis for both our extended software quality model and our practice guidelines.

3.2 Survey on Reproducibility and Quality Requirements

We then designed a survey to assess CSE community members’ attitudes towards software quality and reproducibility [58], using an online questionnaire (the Checkbox survey platform³) to collect our data. To help guide our work, we followed the recommendations of Ghazi et al. for survey-based software engineering research [26]. Following the example of Condori-Fernandez and Lago [19] and using some preliminary survey work with the case study teams [53], we created a survey to ask CSE developers to rank software quality characteristics (as given in the ISO/IEC standard [38]) according to

development teams to identify development bottlenecks and track progress to overcome them. More details on the use of PTCs and their construction can be found in our longer report

³<https://www.checkbox.com/>

their perceived relevance to reproducibility (as defined by NASEM) for their project and asked them to explain their reasoning for their choices. We piloted the survey with friendly participants at Sandia and iteratively refined the wording of our survey instrument before rolling it out to the broader population.

First, we collected demographic information about our survey respondents including employment, level of education, and experience. Participants were then asked to describe a CSE software project that they had contributed to for at least three months and about which they were knowledgeable. Second, respondents were presented with the list of eight software product quality characteristics defined by the ISO/IEC SQuARE standard [38] and were asked to (1) pick quality characteristics they considered relevant to their project and (2) arrange the selected characteristics in order of importance. Third, we presented respondents with a series of questions to measure their beliefs with respect to software quality and reproducibility; these were 5-point Likert scale questions ranging from Strongly Disagree to Strongly Agree [39]. Finally, respondents engaged in a MaxDiff scenario exercise to determine how they would spend hypothetical grant money to focus on improving the quality of their software’s reproducibility.

3.2.1 Survey Recruitment. The survey was advertised first to attendees of the 2023 ACM Conference on Reproducibility and Replacability (ACM REP’23), then within our own research center and lab, to the Slack channel of the United States Research Software Engineer Association (US-RSE)⁴, and informally among contacts in our networks. To increase participation rates, we compensated participants at our laboratory by paying for time spent completing the survey, and, for external participants, we held a \$50 Amazon gift card raffle for ACM REP’23 participants and a \$100 gift card raffle for US-RSE participants. As a form of snowball sampling [8], participants were encouraged to forward the survey to colleagues. The estimated total cost of compensation was \$3000 USD.

We opened the survey in June 2023 and closed it at the end of September 2023, during which time we collected a total of 219 responses. 157 came from our laboratory (15 from targeted interview participants, 142 from the broader lab population), and 62 responses from external respondents.

3.2.2 Interpreting Quality Priority Rankings. The quality priority rankings were converted to Borda counts, which is a positional voting system where the top ranked choice gets $N - 1$ points (where N is the number of choices) and each successive choice gets one less point [22]. We then normalized these counts to sum to one; for example, if respondent that selected performance efficiency, functional suitability, and reliability, we awarded $\{0.38, 0.33, 0.27\}$ normalized points. We note that this treatment of the data assumes that each respondents rankings of priorities are linear in nature, with software qualities becoming evenly less important as the rankings decrease. This is, however, only one of many possible psychometric assumptions that could be made about how the question could be interpreted (e.g., an exponential scale).

3.2.3 MaxDiff. The act of choosing which software quality is most and least important for reproducibility (like the MaxDiff questions

⁴<https://us-rse.org/>

at the end of our survey; see Figure 1) can be viewed as a utility-maximizing decision following Random Utility Theory [15]. We model this data with a multinomial logit (MNL) model [36]. We found the set of weights (or utilities) for each software quality that maximized the posterior (with an L2 regularization term) of the MNL model for the respondents' choices for (1) the whole respondent population (assuming there was a single, shared underlying utility for each quality, depicted in Table 2), (2) each individual respondent (box and whiskers of Figure 2), and (3) separating the respondents into those who said any given quality was or was not important for their project (colored dots in Figure 2).

Most Useful	Make Your Software More...	Least Useful
○	Usable (Can more easily be used by different people; more self-explanatory, easier to learn and use; able to guard against user error; and more accessible and convenient to use)	○
○	Portable (Can be run in more current/future hardware and software environments; more adaptable and easier to install)	●
●	Functionally Suitable (Better able to fulfill all its intended needs; more feature complete, more correct, and able to cover more potential use cases)	○

Figure 1: An example MaxDiff choice set question

3.2.4 Demographics and Project Characteristics. Here we provide a brief overview of the respondent demographics⁵. 90% of respondents came from national laboratories, and almost all responses were from US-based participants. 75.8% reported developing science and/or engineering software in their work, and 61.64% reported using software developed by others. Respondents as a whole skewed toward those with advanced degrees, with 53.42% of respondents having completed a doctoral degree and 25.57% with at least a master's degree, as well as professionally experienced with 41.55% having eleven or more years of professional experience (median 6-10 years). 68.59% of respondents described their projects as being somewhat or very productionized (average 2.85/5, median 3.0/5). Projects varied in team size, with 60.21% of projects having 1-5 team members, 38.74% of projects having 6-15 members, and

17.8% of respondents reported having no one on their team with formal SE training, 30.89% a quarter or less of their team, 16.23% at least half of the team, 14.14% at least three quarters, and 14.14% with all of the team having some amount of formal training.

4 RESULTS

4.1 RQ1: What software development practices facilitate the development and use of reproducible software?

We identified several categories of practice in our case study population (as described in Section 3.1) that they use to increase the reproducibility (and overall quality) of their software throughout

⁵More detail can be found in our longer report [54]

the software development cycle. We present them in alphabetical order.

4.1.1 Automated Testing. Automated testing is the automation of testing software to ensure it meets requirements before being released to developers or users. Testing can cover a wide variety of scales and scopes from unit and integration tests for low- and high-level functions to performance testing for evaluating the scalability of the project. Automated testing was mentioned by 9 out of 9 teams. Some examples of automated testing from our case study population include:

- Project A writes tests to reproduce existing, reputable results (e.g., from professional association standards, textbooks, published paper results, etc.) and automatically tests their code on many different Python and OS versions
- Project D runs a linter and their test suite on each push. If everything passes, they automatically bump the version number and release

Participants attested that testing made their software more reproducible by enhancing the credibility, trustworthiness, and usability of their software. Software cannot be reproducible if it is broken, and testing can quickly identify when and where software is breaking or even directly ensure that specific, important results are being reproduced. Automated testing offloads the critical responsibility of running tests from the developer, ensuring that the tests are always run. It lets developers know quickly when changes (from either new contributions or dependencies) have affected their codebase. The incremental testing encouraged by automated testing can also make it easier to find the basis for new bugs.

“Another large target is to decrease time spent debugging. You are able to spend more time on the science because you have written checks to prevent you from spinning your wheels on what broke [your project]. You save time by spending more time up-front [on setting up automated testing].” - Trinh, Project A

4.1.2 Code Reviews. Code reviews are examinations of existing or new code, often by team members who didn't write the code being reviewed. They are useful for developers to learn the codebase and new techniques, find code errors, and increase code quality. Code review was mentioned by 5 out of 9 teams.

Code reviews encourage developers to write high quality code because they know it will be compared against the agreed upon standards by other members of the team. They increase developer cohesion and confidence when multiple people have signed off saying that any given contribution is sufficient. They also can prompt novel and useful changes to the code being reviewed and teach reviewers new and useful techniques

4.1.3 Development Pipeline. The development pipeline includes the processes to go from committing code with a version control system and automatically testing it (see Sections 4.1.7 and 4.1.1) to deploying code to developers and users. This includes practices relating to change requests, issue tracking, acceptance criteria for merging, etc. Development pipeline was mentioned by 8 out of 9 teams.

Having a development pipeline set up makes it easier to get useful, up-to-date code to users and developers quickly. For example, participants found that issue tracking helped them know when users were struggling and was effective at directing their efforts to areas where they would have high impact. It helped teams to get the relevant information needed to fix important bugs (especially if they implemented issue templates).

4.1.4 Documentation. Documentation communicates how a project works and how it is meant to be used (and not used), for both project contributors and users. This can include written documentation, code comments, code usage examples, tutorials, style guides, and code standards. Documentation was mentioned by 9 out of 9 teams. Some examples of documentation from our case study population include:

- Project F requires documentation for new contributions before they are merged. They also have guidelines on their website for contributors on their practices for library placement, naming, minimum requirements, etc.
- Project I wrote a book on how to use the project and ensure that all examples in the book continue to work via testing
- Project B has docstrings in the code and a wiki with explanations and tutorials (including Jupyter notebooks) outside of the code

Documentation makes the code easier to understand for users and developers, communicates intended scope and use for the software, and encourages developers to write more modular and explainable code. Participants stated that documentation helped with reproducibility by guiding new users, who could start working with the code using tutorials. If new users don't understand how to run the code, they will never be able to reproduce any results.

“Our goal with the documentation is that users are able to successfully install and run Project F without our direct support” - Payne, Project F

Useful documentation increases user trust that the software works as intended and was thoughtfully constructed. It communicates common code usages and what functionalities users can rely on.

4.1.5 Public Processes. Public processes are how the project publicizes their work as well as important code and policy changes. They provide notice and potentially an opportunity for input to the general population. Public processes was mentioned by 5 out of 9 teams. Some examples of public processes from our case study population include:

- Project A has public facing badges for current status of infrastructure, code coverage, and automated testing on the main web page for the project
- Project C maintains a publicly accessible page. This can hold user-friendly documentation, known bugs, current and future development plans, patch notes, and API changes
- Project I responds to questions on commonly used forums such as StackOverflow and Google Groups

Participants stated that public processes make a team's software more reproducible by allowing the developers to align their priorities to those of their stakeholders and users. Users are much more likely to be able to use a given software project and reproduce results that it was used to create if its developers are actively involved with the user community. They also asserted that projects become more reliable and backwards compatible when users can tell whether critical functionalities are changed or removed, and when they can see that the software is being actively maintained to a high standard.

4.1.6 Team Communication. Team communication is how teams exchange information within themselves. This includes communications, online and in-person, formal and informal, and verbal and non-verbal, such as weekly meetings, instant messaging, and emails. Team communication was mentioned by 5 out of 9 teams.

Participants asserted that team communication was essential to keep everyone informed and for taking advantage of the full, diverse expertise of the team. It allows for the transfer of knowledge amongst team members, keeping everyone on the same page and aware of the timeline of future work.

4.1.7 Version Control. Version control is the practice of tracking and managing changes to software. Keeping track of all changes to a codebase allows developers to easily go back to earlier versions of the code when mistakes are made. Version control was mentioned by 9 out of 9 teams. Some examples of version control from our case study population include:

- Project B uses careful branching for feature development to keep the main branch more stable
- Every project uses version control systems (VCS) or source control management (SCM) tools such as Git, SVN, or Mercurial and often pair it with a software publishing tool such as GitLab or GitHub

Participants asserted that version control was fundamental for building modern software.

“The single most important thing we do is use version control.” - Trinh, Project A

Version control allows several developers to be working on the same code simultaneously and aids in merging the changes that they do make together. Participants also attested to version control's usefulness for ensuring reproducibility. When their software breaks, they use version control to roll back to a working version that was making reproducible results until they can fix the new issues.

RQ1 Summary: Based on the synthesis of our case study interviews, there are several categories of software engineering practice that are agreed to contribute to computational reproducibility. Among these, automated testing, version control, and documentation stand out as being universally agreed upon to be useful practices.

4.2 RQ2: Beliefs about Software Quality and Reproducibility

With our survey we investigated CSE developers beliefs about the relationship between software quality and reproducibility, and whether they feel willing and able to improve the quality of their software to benefit reproducibility. This information is important for non-development stakeholders to understand what is required if they want reproducibility for their software project. Because we are simultaneously considering many different measures of beliefs, we apply the Bonferroni correction to mitigate the multiple comparisons problem [11]. Without correction, the likelihood of incorrectly rejecting a null hypothesis (i.e., making a Type I error) increases with the number of hypotheses being tested. Thus, when we report on correlations, those with significance $p \leq \alpha/18$ are highlighted with a check mark (✓).

CSE developers overwhelmingly agreed that science and engineering software needs to be reproducible to be useful (94.52% agree), and that reproducibility enables scientific progress by enabling collaboration and improving the ability to build upon previous findings (97.71% agree). 86.3% agreed that high-quality software is more likely to yield reproducible results than low-quality software (10.05% neither agreed nor disagreed). 63.01% went even further, claiming that reproducibility was more important than the functionality or performance of their software.

We included three questions to measure the intent or willingness of participants to act on the belief that software quality and reproducibility were related. If it would help ensure reproducibility, 91.1% said they were willing to invest extra effort in software quality, 63.16% were willing to stand up complex solutions (e.g., containerizing codes to support reproducibility), and 50.79% were willing to sacrifice a software quality (e.g., limiting its capabilities or performance).

While 9 out of 10 respondents said they would be willing to spend extra effort on improving software quality to better support reproducibility, only 57.59% of respondents felt that they were able to quickly identify and take action on opportunities to improve their code in this way. Those who felt their project(s) had implemented good practices were more likely to report that managing reproducibility did not hinder their productivity ($\checkmark r = 0.417, p = 3.848e - 08$), as were those who felt their stakeholders had a good understanding of the time, cost, and effort needed to ensure reproducibility ($\checkmark r = 0.343, p = 8.371e - 06$). 66.21% of respondents would like to see common standards of practices for reproducibility among CSE projects, and 90.32% want more training and professional development opportunities to learn about best practices for reproducibility.

Finally, we also measured teams' desires for duration of reproducibility. For most teams, the need to reproduce a particular set of results goes down over time, if only because results from a past study are eventually superseded by better results produced by improved algorithms. 94.24% of respondents with software projects stated it was important or very important for results from their software to remain reproducible months after the original execution, 64.4% for years, and 26.18% for decades.

RQ2 Summary: The prevailing view of our respondents is that having software that provides reproducible results (at least for months or years into the future) is essential, that software quality plays a role in ensuring those results, and that respondents would be willing to spend more time and effort on improving software quality for this reason. We also noted that four in ten respondents did not feel ready and able to quickly take action on their codebase to improve reproducibility, and nine out of ten reported wanting more training and professional opportunities to learn more about reproducibility best practices.

4.3 RQ3: Perceived Contributions to Reproducibility of Software Product Qualities

A key goal of our study is to better understand how software product quality characteristics are understood to contribute to the reproducibility of CSE software and therefore what qualities should be focused on to make the overall software more reproducible. In the MaxDiff portion of the survey, we explored quality requirements by asking respondents to explicitly choose between different characteristics, asking which would contribute to reproducibility more. Each survey respondent filled out ten choice sets on the perceived relative value of different software product quality characteristics to reproducibility. As shown in Table 2, (1) maintainability, reliability, usability, and functional suitability were most frequently ranked as useful across choice sets; (2) respondents were divided on the usefulness of portability and compatibility; and (3) performance efficiency and security were most likely to be ranked as least useful.

A potential source of bias is respondents over-valuing software qualities that they are more familiar with or that are important for their current or past projects. Because we also collected data on the overall quality priorities of projects, we can account for these differences. In Figure 2 we compare the estimated MaxDiff utilities among people who did and did not rank a given software product quality characteristic as being important to their project (irrespective of reproducibility considerations). A given software quality was around twice as likely to be picked as more important than others for reproducibility by participants whose project prioritize that quality. Regardless of respondent project's software quality priorities, maintainability, reliability, and usability remain relatively important, and performance efficiency and security remain relatively unimportant.

Our final question is whether respondents already prioritize reproducibility-aligned characteristics on their software projects. In Table 3, we present statistics on how frequently different characteristics were selected as being relevant to respondents' projects and how frequently each characteristic was selected as the most relevant characteristic for their project. Looking at these results in aggregate, however, only provides us limited insight into how projects differ from one another.

To break down respondents into smaller groups, we ran k-means clustering [51] on the normalized software quality priority Borda counts. We chose to separate the respondents into three clusters as

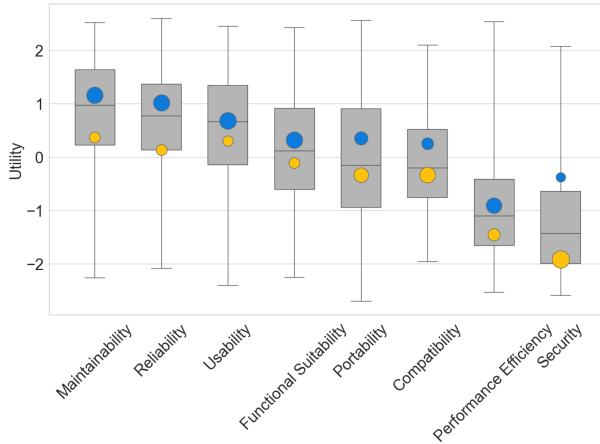


Figure 2: A box and whisker plot illustrating the perceived contribution to reproducibility for each software quality. It shows the distribution (specifically each quartile) of utility scores per-person for different software product quality characteristics, regardless of reproducibility priorities. In addition, we show how project priorities affect utility estimates. The higher blue dot indicates the utility of each characteristic according to respondents who ranked that characteristic as important for their project (regardless of reproducibility priorities), and the lower orange dot indicates the utility by those who did not rank that characteristic as being important. The relative size of the dots reflects the percentage of respondents who did or did not consider a given characteristic as being important.

Table 2: Tabulated results for the MaxDiff exercise, showing (1) how frequently each software product quality characteristic was selected as most useful and least useful across all choice sets and (2) the utility scores derived from the multinomial logit model based on respondents' selections.

Product Quality Characteristic	Most Useful (%)	Least Useful (%)	Estimated Utility
Maintainability	21.52%	4.14%	0.981 ↑↑
Reliability	18.59%	4.87%	0.807 ↑↑
Usability	17.64%	6.44%	0.613 ↑↑
Functional Suitability	13.77%	9.63%	0.233 ↑↑
Portability	12.72%	12.93%	-0.027 ⇌
Compatibility	9.11%	11.78%	-0.081 ⇌
Performance Efficiency	3.61%	23.14%	-1.034 ↓↓
Security	1.99%	26.02%	-1.492 ↓↓

this was the smallest number of clusters such that the gap statistic [69] was within one standard deviation of the gap at a higher number of clusters. The software quality priorities for respondents associated with each of the three respondent profiles (A, B, and C)

Table 3: Tabulated results for the respondents' software quality priorities, showing how frequently each software product quality characteristic was selected as relevant for their project and how frequently each quality was picked as the most relevant for their project.

Product Quality Characteristic	Relevant	Most Relevant
Functional Suitability	79.57%	34.95%
Usability	83.87%	22.58%
Reliability	78.49%	12.90%
Maintainability	79.57%	9.14%
Performance Efficiency	71.51%	7.53%
Security	20.97%	7.53%
Portability	48.39%	3.23%
Compatibility	42.47%	2.15%

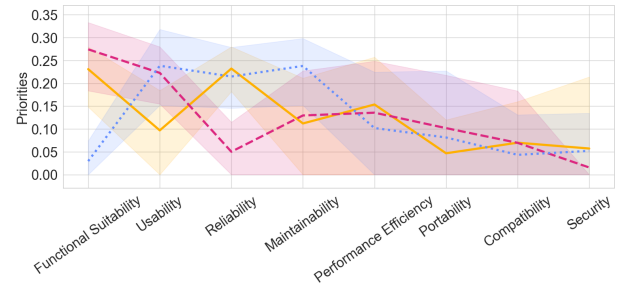


Figure 3: Line plots showing the normalized Borda count software quality priorities for members of the three respondent profiles identified with k-means clustering. The center lines show the k-means cluster centers and the bands show the 16th to 84th percentiles. The solid orange, dashed magenta, and dotted blue correspond to profiles A, B, and C.

are shown in Figure 3. After identifying the three respondent profiles, we searched for any correlations between profile membership and survey responses.

- Respondents associated with profile A (71 respondents) are significantly more likely ($p < \alpha/18$) to care about reliability ($p = <1e-4$), and not care about usability, portability, or maintainability ($p = <1e-4$ to 0.0012). They are slightly more likely ($\alpha/18 < p < \alpha$) to be more experienced ($p = 0.0077$) and have larger teams ($p = 0.0302$).
- Respondents associated with profile B (67 respondents) are significantly more likely to identify as researchers ($p = 0.0006$), have a PhD ($p = 0.0019$), and care about functional suitability ($p = <1e-4$) and usability ($p = 0.0001$) but *not* reliability ($p = <1e-4$). They are slightly more likely to believe that their stakeholders *do not* value reproducibility ($p = 0.0046$) and that security is not important for reproducibility ($p = 0.0093$).

- Respondents associated with profile C (48 respondents) are significantly more likely to care about maintainability, usability, and reliability ($p = 0.0001$ – $<1e-4$), but *not* functional suitability ($p = <1e-4$). They are slightly more likely to identify as software developers ($p = 0.0308$) and *not* researchers ($p = 0.0429$), have a Bachelor's and/or a Master's degree as opposed to a PhD ($p = 0.0122$), and to not care about performance ($p = 0.0153$). They also are slightly more likely to believe that their stakeholders *do* value reproducibility ($p = 0.043$).

We observe that while respondents in profile C exhibit a strong focus on maintainability, usability, and reliability which makes them more aligned to qualities that are believed to aid in ensuring software reproducibility, others have software quality priorities that are not as well-aligned with qualities that contribute to reproducibility and thus will need to make special efforts to ensure that they make and keep their project reproducible.

RQ3 Summary: The consensus from the MaxDiff exercise is that (1) maintainability, reliability, usability, and (to a lesser extent) functional suitability are generally considered useful for reproducibility, (2) opinions were mixed on portability and compatibility, and (3) performance efficiency and security are believed to be least useful. CSE developers with projects we surveyed reported a wide spread of product quality priorities. Among them we note that one in four respondents exhibited a strong focus on maintainability, usability, and reliability, making them well aligned with their reproducibility priorities.

5 LIMITATIONS OF THE STUDY

There are several threats to the validity of this study. This study, by design, primarily focused on Members of the Workforce (MOWs) at Sandia National Laboratories. While this was the intent of the study, it limits our ability to generalize our findings. We mediated this by releasing our survey to external participants outside of MOWs; however, in particular for findings from the case study, the results represent the ecosystem and its priorities. Additionally, survey results still select heavily from national laboratories (75% of respondents), limiting our ability to make statements about other institutional types.

Since Sandia National Laboratories deals with high-risk and mission-critical applications that are essential to national security, priorities in this realm will be different from low-risk academic or consumer driven applications. As an example, we found through the case studies that those projects that were able to have publicly open policies and procedures did so and felt that they effected better reproducibility; however, sensitive applications are, by nature, not able to utilize open-source policies. As a result, we assume that an extension of this study outside a national security laboratory will show different results.

Finally, A notable limitation of the case studies is that useful practice categorization was performed post-hoc, so that interviewers did not necessarily ask targeted questions about each practice. While this expanded the range of possible practices to be considered, this did mean that teams were not necessarily prompted to comment on

a particular practice. Thus, a failure to mention a particular practice does not necessarily mean that a team does not employ it.

6 DISCUSSION

In addition to the results relating to our primary research questions on the relationship between software qualities and reproducibility and practices that enable it, the results from the survey and case studies revealed some cross-cutting themes and barriers preventing teams from keeping their projects reproducible that are especially important for project stakeholders to understand. The cross-cutting themes revealed how teams used best practices to communicate the intended use of their software, learn from their peers to adopt new practices and tools, benefit from having a wide variety of expertise, and believe that practices should implemented at logical times. The barriers that were found were that there is a lack of experience, training, and institutional support for reproducibility efforts, and that (at Sandia) reproducibility is hindered by non-standardized software development pipelines and ecosystems and inconsistent reproducibility requirements

6.1 Discovered Themes

6.1.1 Communicating Intended Use. A cross-cutting theme for many of the case study projects was the idea of “intended use.” When it comes to reproducibility, projects can only provide it if their users adhere to policies and published material that communicate the software’s intended use. This both manages expectations of the users and provides a basis to which projects can point as their promise for reproducible behavior. Without a clear delineation of supported and expected interactions, reproducibility is near impossible to promise.

“Documentation prevents people from using the software in totally unexpected ways. We have a lot of commands - documenting them lets other people use them. Documentation narrows the state space in which code can occupy.” - Darby, Project H

6.1.2 Learning from Others. People are more likely to listen to the recommendations of someone they know and trust [63]. This is equally as true for CSE teams as it is for purchasing commercial products [41]. Raybourn et al. conducted a discussion session as part of the Collegeville 2021 Workshop on scientific software teams⁶ regarding incentivizing software quality practices. As noted in their report: “The biggest takeaway from the Collegeville teatime discussion, in fact, was that the highest likelihood for change comes from external sources and a main, influential advocate. Participants expressed emphatically that word-of-mouth recommendations for practices and processes easily holds the biggest sway for any substantive change” [62].

The case study interviews strongly supported this claim. In fact, seven of the nine case study projects stated during their specific practice interviews that they had learned how to implement a particular practice from a trusted colleague or software package similar to their own.

⁶<https://collegeville.github.io/CW21/>

The reasons for this were varied: (1) lack of time to learn a new practice on their own; (2) trust in experienced coworkers or mentors; and (3) trust in the expertise of other tools that appeared to be of good quality.

6.1.3 Necessity for Multiple Domain Experts. From design to development, from documentation to review, a universal consensus across the projects was that more, varied expertise was better. This is unsurprising within the multi-disciplinary ecosystem of modern CSE projects. In particular, however, CSE projects require more expertise in software engineering (SWE) [55]. Projects within the case study universally stated the importance of accessibility to multiple domain experts, particularly within SWE.

“We basically knew nothing going in. We thought we should have testing... But that was about it. We had to figure out how to use pytest effectively, how to set up the GitLab runners on Sandia resources. It was a convoluted mess (and still is, but less so). We didn’t know anything about any of the infrastructure needed.” - Châu, Project D

Similarly, with so many domains represented in modern CSE software, reviews and experience are needed to ensure quality.

6.1.4 Right-size Solutions. When it comes to applying practices to improve quality, the general belief is that there is a right time to do so. Certain practices, such as version control, should be implemented immediately as a basic foundational requirement for reproducibility. Other practices should be introduced at logical times, generally in response to stimuli, such as a specific barrier or need. For example, user documentation is not necessary until a software is intended to be released to users.

6.2 Barriers to Effectively Integrating Reproducibility Into Software Projects

6.2.1 Lack of Experience and Training. In the survey data, it is indicated that over half of the projects surveyed are in some form of productionization. Yet, for almost half of all teams, 50% or less of the team’s experience is ranked as “significant” or “high.” This indicates that many teams consist mainly of developers who do not have extensive experience to run production software projects. 90% of survey respondents indicate that there should be training and development opportunities to help people learn more about best practices for reproducibility. However, if less experienced developers are trying to run production software projects, they may prioritize more foundational software skills over more advanced reproducibility training. Therefore, successful approaches to reproducibility training should *integrate foundational software engineering techniques* to ensure that team members are incentivized to attend.

6.2.2 Lack of Institutional Support. Case studies indicated that there was a lack of support from stakeholders, notably leadership and funders, for improving reproducibility as a project priority. The assertion that leadership is less supportive than domain communities is supported additionally by survey data (see Section 4.2). Some noted that, at Sandia National Laboratories, there is no funding mechanism for improving reproducibility.

6.2.3 Non-standardized Ecosystems. Multiple case study projects referred to difficulties in the infrastructure and tools available in the software development ecosystem within Sandia National Laboratories. For example, before the release of GitHub Actions⁷, open-source projects had to rely on un- or lightly-supported internal toolchains for automatic testing. Some teams used Jenkins⁸; some used GitLab CI/CD⁹. Most teams had to set up custom software license servers for the software they relied on for their individual projects. Setting up a software ecosystem for each individual project’s needs requires an immense amount of time and expertise, but they are necessary to effectively coordinate Automated Testing, Documentation, and Development Pipelines. This indicates that a non-standardized ecosystem has resulted in a very serious barrier to more reproducible software.

6.2.4 Diverse Conceptions of Reproducibility. Reproducibility ideas and definitions are not consistent across teams. This may be driven by heterogeneous needs; for example, high-consequence work will demand an extreme amount of rigor and reliability that exploratory work will not. This makes it implausible to devise a single approach which will meet every team’s needs.

7 CONCLUSION

From the case studies, there were several practices that were highlighted as being very useful. Automated testing, documentation, and version control were universally remarked upon as useful practices. This is not surprising, given how often these practices are recommended for software engineering in general. Documentation appears to establish the bounds of the software scope and how it should be used, creating a shared expectation for software behavior. Automated testing helps to ensure that this expectation is met by continuously evaluating the software as it is changed, upholding the guarantees within the documentation. The development process then establishes a process by which the development team can manage changes. This suggests that effective frameworks for enhancing reproducibility in software projects should contain three components:

- A means of communicating to users and developers how the software is intended to be used, and what its limitations are.
- A means of verifying that the documentation’s guarantees are upheld.
- A process by which the relationship between the promises of the documentation and the results of the testing remain synchronized.

Other practices were less commonly referred to, such as public processes, but almost every practice was utilized by a majority of teams, suggesting that there is relative agreement between teams about what practices are useful.

From the survey, we found that among CSE professionals, there is an increasingly popular belief that adopting modern software development practices could enable the production of better software and better science. As Jimenez et al. state, “software development best

⁷<https://docs.github.com/en/actions>

⁸<https://www.jenkins.io/>

⁹<https://docs.gitlab.com/ee/ci/>

practices promote better quality software, and better quality software improves the reproducibility and reusability of research”[40]. Among those we surveyed, the majority of respondents believed that high-quality software tended to be more reproducible, and reported they would be willing to invest more time and effort into improving the quality of their software if doing so would improve reproducibility. Even with the barriers that exist, people want to make their software reproducible. Next, we found general agreement that certain characteristics of product quality are beneficial for reproducibility: maintainability, reliability, and usability. While some software developers will have software quality priorities that are well aligned with reproducibility, the majority of people will have to make deliberate trade-offs to ensure that the software that they are building becomes, and remains, reproducible.

As noted in Section 5, interviewers did not necessarily ask targeted questions about each identified practice, and the results from this study are heavily skewed towards Sandia National Laboratories MOWs’ beliefs and priorities with respect to reproducibility. Future work should address some of these practices with a more targeted approach. For example, Team Communication and Code Reviews are often part of a Development Pipeline; however, they are mentioned by relatively fewer teams. Future work should investigate whether there are other practices that are so foundational that teams just assume that they should be practicing them. Identifying which practices fall under this category would help indicate which practices are already widespread. Future work should be extended to a larger basis of participants and seek how to quantify the contribution of different practices to achieving reproducibility. For example, all projects referred to automated testing as crucial for ensuring reproducibility. However, it is unclear *how much* or *what* testing is necessary. Papers have been written comparing testing efficacy for finding defects across software types (c.f., [7]); however, this is an understudied topic in CSE.

These preliminary findings suggest opportunities for future investigation, such as into training, tools, and practices that could enable better software quality engineering for CSE software products to support teams’ reproducibility requirements. We also noted a broader need for more education and awareness among stakeholders regarding the time, cost, and effort that ensuring reproducibility requires. Our hope is that our findings help promote dialogue between CSE developers and the customers who fund them, particularly around the need to explicitly invest in software quality rather than just the science that the software produces; our results suggest that these things are, in fact, intimately related.

REFERENCES

- [1] 2020. Artifact Review and Badging Version 1.1. <https://www.acm.org/publications/policies/artifact-review-and-badging-current/>
- [2] Carl S Adorf, Vyas Ramasubramani, Joshua A Anderson, and Sharon C Glotzer. 2018. How to professionally develop reusable scientific software—and when not to. *Computing in Science & Engineering* 21, 2 (2018), 66–79.
- [3] Andrés F Aguirre, Ángela Villareal-Freire, Rosa Gil, and César A Collazos. 2017. Extending the concept of user satisfaction in e-learning systems from ISO/IEC 25010. In *International Conference of Design, User Experience, and Usability*. Springer, 167–179.
- [4] Lorena A Barba. 2019. Praxis of reproducible computational science. *Computing in Science & Engineering* 21, 1 (2019), 73–78.
- [5] Rob Baxter, N Chue Hong, Dirk Gorissen, James Hetherington, and Ilian Todorov. 2012. The research software engineer. In *Digital Research Conference, Oxford*. 1–3.
- [6] Marijan Beg, Juliette Taka, Thomas Kluyver, Alexander Kononov, Min Ragan-Kelley, Nicolas M Thiéry, and Hans Fangohr. 2021. Using Jupyter for reproducible scientific workflows. *Computing in Science & Engineering* 23, 2 (2021), 36–46.
- [7] Jonathan Bell, Christian Murphy, and Gail Kaiser. 2015. Metamorphic runtime checking of applications without test oracles. *CrossTalk* 28, 2 (2015).
- [8] Patrick Biernacki and Dan Waldorf. 1981. Snowball sampling: Problems and techniques of chain referral sampling. *Sociological methods & research* 10, 2 (1981), 141–163.
- [9] Carl Boettiger. 2015. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review* 49, 1 (2015), 71–79.
- [10] Carl Boettiger, Scott Chamberlain, Edmund Hart, and Karthik Ram. 2015. Building software, building community: lessons from the rOpenSci project. *Journal of open research software* 3, 1 (2015).
- [11] Carlo Bonferroni. 1936. Teoria statistica delle classi e calcolo delle probabilità. *Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze* 8 (1936), 3–62.
- [12] Massimiliano Bonomi. 2019. Promoting transparency and reproducibility in enhanced molecular simulations. *Nature methods* 16, 8 (2019), 670–673.
- [13] Alys Brett, Michael Croucher, Robert Haines, Simon Hettrick, James Hetherington, Mark Stillwell, and Claire Wyatt. 2017. Research software engineers: state of the nation report 2017. (2017).
- [14] Coral Calero, María Ángeles Moraga, Manuel F Bertoa, and Leticia Duboc. 2014. Quality in Use and Software Greenability. In *RE4SuSy@ RE*. 28–36.
- [15] Ennio Cascetta. 2009. *Random Utility Theory*. Springer US, Boston, MA, 89–167. https://doi.org/10.1007/978-0-387-75857-2_3
- [16] Kyle Chard, Niall Gaffney, Matthew B Jones, Kacper Kowalik, Bertram Ludäscher, Jarek Nabrzyski, Victoria Stodden, Ian Taylor, Matthew J Turk, and Craig Willis. 2019. Implementing computational reproducibility in the Whole Tale environment. In *Proceedings of the 2nd International Workshop on Practical Reproducible Evaluation of Computer Systems*. 17–22.
- [17] Adam S Charles, Benjamin Falk, Nicholas Turner, Talmo D Pereira, Daniel Tward, Benjamin D Pedigo, Jaewon Chung, Randal Burns, Satrajit S Ghosh, Justus M Kebschull, et al. 2020. Toward Community-Driven big Open brain science: open big data and tools for structure, function, and genetics. *Annual review of neuroscience* 43 (2020), 441–464.
- [18] Jeremy Cohen, Daniel S Katz, Michelle Barker, Neil Chue Hong, Robert Haines, and Caroline Jay. 2020. The four pillars of research software engineering. *IEEE Software* 38, 1 (2020), 97–105.
- [19] Nelly Condori-Fernandez and Patricia Lago. 2018. Characterizing the contribution of quality requirements to software sustainability. *Journal of systems and software* 137 (2018), 289–305.
- [20] Paolo Di Tommaso, Maria Chatzou, Evan W Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. Nextflow enables reproducible computational workflows. *Nature biotechnology* 35, 4 (2017), 316–319.
- [21] Anshu Dubey, Jared O’Neal, Klaus Weide, and Saurabh Chawdhary. 2020. Distillation of best practices from refactoring flash for exascale. *SN Computer Science* 1, 4 (2020), 1–9.
- [22] Peter Emerson. 2013. The original Borda count and partial voting. *Social Choice and Welfare* 40 (2013), 353–358.
- [23] Stephen J Ezell and Robert D Atkinson. 2016. The vital importance of high-performance computing to US competitiveness. *Information Technology and Innovation Foundation* 28 (April 2016).
- [24] Juliana Freire, Philippe Bonnet, and Dennis Shasha. 2012. Computational reproducibility: state-of-the-art, challenges, and database research opportunities. In *Proceedings of the 2012 ACM SIGMOD international conference on management of data*. 593–596.
- [25] Lilia García-Mundo, Marcela Genero, and Mario Piattini. 2015. Towards a construction and validation of a serious game product quality model. In *2015 7th International Conference on Games and Virtual Worlds for Serious Applications (VS-Games)*. IEEE, 1–8.
- [26] Ahmad Nauman Ghazi, Kai Petersen, Sri Sai Vijay Raj Reddy, and Harini Nekkanti. 2018. Survey research in software engineering: Problems and mitigation strategies. *IEEE Access* 7 (2018), 24703–24718.
- [27] BG Glaser and AL Strauss. 1967. The discovery of grounded theory: strategies for qualitative research. (1967).
- [28] Jeremy Goecks, Anton Nekrutenko, and James Taylor. 2010. Galaxy: a comprehensive approach for supporting accessible, reproducible, and transparent computational research in the life sciences. *Genome biology* 11, 8 (2010), 1–13.
- [29] Jose Luis González, Roberto García, Josep Maria Brunetti, Rosa Gil, and Juan Manuel Gimeno. 2012. SWET-QUM: a quality in use extension model for semantic web exploration tools. In *Proceedings of the 13th International Conference on Interacción Persona-Ordenador*. 1–8.
- [30] Samuel Grayson, Darko Marinov, Daniel S Katz, and Reed Milewicz. 2023. Automatic Reproduction of Workflows in the Snakemake Workflow Catalog and nf-core Registries. In *Proceedings of the 2023 ACM Conference on Reproducibility and Replicability*. 74–84.

- [31] Odd Erik Gundersen. 2021. The fundamental principles of reproducibility. *Philosophical Transactions of the Royal Society A* 379, 2197 (2021), 20200210.
- [32] Michael Haider, Michael Riesch, and Christian Jirauschek. 2021. Realization of best practices in software engineering and scientific writing through ready-to-use project skeletons. *Optical and Quantum Electronics* 53, 10 (2021), 1–17.
- [33] Wilhelm Hasselbring, Leslie Carr, Simon Hettrick, Heather Packer, and Thanassis Tiropanis. 2020. Open source research software. *Computer* 53, 8 (2020), 84–88.
- [34] Michael A Heroux, Elsa Gonsiorowski, Rinku Gupta, Reed Milewicz, J David Moulton, Gregory R Watson, Jim Willenbring, Richard J Zamora, and Elaine M Raybourn. 2020. Lightweight software process improvement using productivity and sustainability improvement planning (PSIP). In *Tools and Techniques for High Performance Computing: Selected Workshops, HUST, SE-HER and WIHPC, Held in Conjunction with SC 2019, Denver, CO, USA, November 17–18, 2019, Revised Selected Papers* 6. Springer, 98–110.
- [35] Alexandre Hocquet and Frédéric Wieber. 2021. Epistemic issues in computational reproducibility: software as the elephant in the room. *European Journal for Philosophy of Science* 11 (2021), 1–20.
- [36] Sawtooth Software Inc. 2020. The MaxDiff System Technical Paper. <https://sawtoothsoftware.com/resources/technical-papers/maxdiff-technical-paper>
- [37] Darrel C Ince, Leslie Hatton, and John Graham-Cumming. 2012. The case for open computer programs. *Nature* 482, 7386 (2012), 485–488.
- [38] ISO/IEC. 2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models (25010:2011). <https://www.iso.org/standard/35733.html>.
- [39] Andrew T Jebb, Vincent Ng, and Louis Tay. 2021. A review of key Likert scale development advances: 1995–2019. *Frontiers in psychology* 12 (2021), 637547.
- [40] Rafael C Jiménez, Mateusz Kuzak, Monther Alhamdoosh, Michelle Barker, Bérénice Batut, Mikael Borg, Salvador Capella-Gutierrez, Neil Chue Hong, Martin Cook, Manuel Corpas, et al. 2017. Four simple recommendations to encourage best practices in research software. *F1000Research* 6 (2017).
- [41] Daniel S Katz, Lois Curfman McInnes, David E Bernholdt, Abigail Cabunoc Mayes, Neil P Chue Hong, Jonah Duckles, Sandra Gesing, Michael A Heroux, Simon Hettrick, Rafael C Jimenez, et al. 2018. Community organizations: changing the culture in which research software is developed and sustained. *Computing in Science & Engineering* 21, 2 (2018), 8–24.
- [42] Daniel S Katz, Kyle E Niemeyer, and Arfon M Smith. 2018. Publish your software: introducing the journal of open source software (JOSS). *Computing in Science & Engineering* 20, 3 (2018), 84–88.
- [43] Louise H Kellogg, Lorraine J Hwang, Rene Gassmüller, Wolfgang Bangerth, and Timo Heister. 2018. The role of scientific communities in creating reusable software: Lessons from geophysics. *Computing in Science & Engineering* 21, 2 (2018), 25–35.
- [44] Diane F Kelly. 2007. A software chasm: Software engineering and scientific computing. *IEEE software* 24, 6 (2007), 120–119.
- [45] Johannes Köster and Sven Rahmann. 2012. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics* 28, 19 (2012), 2520–2522.
- [46] Bojana Koteska, Anastas Mishev, and Ljupco Pejov. 2018. Quantitative measurement of scientific software quality: definition of a novel quality model. *International Journal of Software Engineering and Knowledge Engineering* 28, 03 (2018), 407–425.
- [47] Hiroshi Kuwajima and Fuyuki Ishikawa. 2019. Adapting SQuaRE for quality assessment of artificial intelligence systems. In *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 13–18.
- [48] Benjamin D Lee. 2018. Ten simple rules for documenting scientific software.
- [49] Randall J LeVeque, Ian M Mitchell, and Victoria Stodden. 2012. Reproducible research for scientific computing: Tools and strategies for changing the culture. *Computing in Science & Engineering* 14, 4 (2012), 13–17.
- [50] Jimmy Lin and Qian Zhang. 2020. Reproducibility is a process, not an achievement: The replicability of IR reproducibility experiments. In *European Conference on Information Retrieval*. Springer, 43–49.
- [51] James MacQueen et al. 1967. Some methods for classification and analysis of multivariate observations. In *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, Vol. 1. Oakland, CA, USA, 281–297.
- [52] G Uma Maheswari and VV Rama Prasad. 2011. Optimized software quality assurance model for testing scientific software. *International Journal of Computer Applications* 36, 7 (2011), 37–46.
- [53] Reed Milewicz and Miranda Mundt. 2023. Towards Evidence-Based Software Quality Practices for Reproducibility: Preliminary Results and Research Directions. In *Proceedings of the 2023 ACM Conference on Reproducibility and Replicability* (Santa Cruz, CA, USA) (ACM REP '23). Association for Computing Machinery, New York, NY, USA, 85–88. <https://doi.org/10.1145/3589806.3600040>
- [54] Reed Milewicz, Miranda Mundt, Christian Gilbertson, and Joshua Teves. 2023. *Evidence-Based Software Quality Practices for Computational Reproducibility*. Technical Report SAND2023-10506R.
- [55] Miranda Mundt and Reed Milewicz. 2022. Working in Harmony: Towards Integrating RSEs into Multi-Disciplinary CSE Teams. *arXiv preprint arXiv:2201.04010* (2022).
- [56] National Academies of Sciences, Engineering, and Medicine et al. 2019. *Reproducibility and replicability in science*. National Academies Press.
- [57] Daniel Nüst, Vanessa Sochat, Ben Marwick, Stephen J Eglen, Tim Head, Tony Hirst, and Benjamin D Evans. 2020. Ten simple rules for writing Dockerfiles for reproducible data science. , e1008316 pages.
- [58] Shari Lawrence Pfleeger and Barbara A Kitchenham. 2001. Principles of survey research: part 1: turning lemons into lemonade. *ACM SIGSOFT Software Engineering Notes* 26, 6 (2001), 16–18.
- [59] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. 2019. A large-scale study about quality and reproducibility of jupyter notebooks. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 507–517.
- [60] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. 2021. Understanding and improving the quality and reproducibility of Jupyter notebooks. *Empirical Software Engineering* 26, 4 (2021), 65.
- [61] F Queiroz, R Silva, J Miller, S Brockhauser, and H Fangohr. 2017. Good Usability Practices in Scientific Software Development. In *Proceedings of the Workshop on Sustainable Software for Science: Practice and Experiences (WSSSPE5. 1)*. WSSSPE.
- [62] Elaine Raybourn, Reed Milewicz, and Miranda Mundt Brandenburg. 2022. Incentivizing Adoption of Software Quality Practices. *SAND2022-1691* (2 2022). <https://doi.org/10.2172/1845193>
- [63] Everett M Rogers, Arvind Singhal, and Margaret M Quinlan. 2014. Diffusion of innovations. In *An integrated approach to communication theory and research*. Routledge, 432–448.
- [64] Per Runeson, Martin Host, Austen Rainer, and Bjorn Regnell. 2012. *Case study research in software engineering: Guidelines and examples*. John Wiley & Sons.
- [65] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. 2013. Ten simple rules for reproducible computational research. *PLoS computational biology* 9, 10 (2013), e1003285.
- [66] Frank Philip Seth, Erja Mustonen-Ollila, Ossi Taipale, and Kari Smolander. 2015. Software quality construction in 11 companies: an empirical study using the grounded theory. *Software Quality Journal* 23 (2015), 627–660.
- [67] Victoria Stodden, Christophe Hurlin, and Christophe Pérignon. 2012. RunMyCode.org: A novel dissemination and collaboration platform for executing published computational results. In *2012 IEEE 8th International Conference on E-Science*. IEEE, 1–8.
- [68] Victoria Stodden, Sheila Miguez, and Jennifer Seiler. 2015. Researchcompendia.org: Cyberinfrastructure for reproducibility and collaboration in computational science. *Computing in Science & Engineering* 17, 1 (2015), 12–19.
- [69] Robert Tibshirani, Guenther Walther, and Trevor Hastie. 2001. Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 63, 2 (2001), 411–423.
- [70] Ana Trisovic, Matthew K Lau, Thomas Pasquier, and Mercè Crosas. 2022. A large-scale study on research code quality and execution. *Scientific Data* 9, 1 (2022), 60.
- [71] Nicole A Vasilevsky, Jessica Minnier, Melissa A Haendel, and Robin E Champieux. 2017. Reproducible and reusable research: are journal data sharing policies meeting the mark? *PeerJ* 5 (2017), e3208.
- [72] Craig Willis and Victoria Stodden. 2020. Trust but verify: How to leverage policies, workflows, and infrastructure to ensure computational reproducibility in publication. (2020).
- [73] Nebi Yilmaz and Ayça Kolukisa Tarhan. 2020. Meta-models for software quality and its evaluation: a systematic literature review. In *International Workshop on Software Measurement and the 15th International Conference on Software Process and Product Measurement, Mexico*.